1.8.4 The Model of Run Time

Once the operating system binary has been linked, the computer can be rebooted and the new operating system started. Once running, it may dynamically load pieces that were not statically included in the binary such as device drivers and file systems. At run time, the operating system may consist of multiple segments, for the text (the program code), the data, and the stack. The text segment is normally immutable, not changing during execution. The data segment starts out at a certain size and initialized with certain values, but it can change and grow as need be. The stack is initially empty but grows and shrinks as functions are called and returned from. Often the text segment is placed near the bottom of memory, the data segment just above it, with the ability to grow upward, and the stack segment at a high virtual address, with the ability to grow downward, but different systems work differently.

In all cases, the operating system code is directly executed by the hardware, with no interpreter and no just-in-time compilation, as is normal with Java.

1.9 RESEARCH ON OPERATING SYSTEMS

Computer science is a rapidly advancing field and it is hard to predict where it is going. Researchers at universities and industrial research labs are constantly thinking up new ideas, some of which go nowhere but some of which become the cornerstone of future products and have massive impact on the industry and users. Telling which is which turns out to be easier to do in hindsight than in real time. Separating the wheat from the chaff is especially difficult because it often takes 20 to 30 years from idea to impact.

For example, when President Dwight Eisenhower set up the Dept. of Defense's Advanced Research Projects Agency (ARPA) in 1958, he was trying to keep the Army from killing the Navy and the Air Force over the Pentagon's research budget. He was not trying to invent the Internet. But one of the things ARPA did was fund some university research on the then-obscure concept of packet switching, which led to the first experimental packet-switched network, the ARPANET. It went live in 1969. Before long, other ARPA-funded research networks were connected to the ARPANET, and the Internet was born. The Internet was then happily used by academic researchers for sending email to each other for 20 years. In the early 1990s, Tim Berners-Lee invented the World Wide Web at the CERN research lab in Geneva and Marc Andreesen wrote a graphical browser for it at the University of Illinois. All of a sudden, the Internet was full of twittering teenagers. President Eisenhower is probably rolling over in his grave.

Research in operating systems has also led to dramatic changes in practical systems. As we discussed earlier, the first commercial computer systems were all batch systems, up until M.I.T. invented general-purpose timesharing in the early

1960s. Computers were all text-based until Doug Engelbart invented the mouse and the graphical user interface at Stanford Research Institute in the late 1960s. Who knows what will come next?

In this section, and in comparable sections throughout the book, we will take a brief look at some of the research in operating systems that has taken place during the past 5–10 years, just to give a flavor of what might be on the horizon. This introduction is certainly not comprehensive. It is based largely on papers that have been published in the top research conferences because these ideas have at least survived a rigorous peer review process in order to get published. Note that in computer science—in contrast to other scientific fields—most research is published in conferences, not in journals. Most of the papers cited in the research sections were published by either ACM, the IEEE Computer Society, or USENIX and are available over the Internet to (student) members of these organizations. For more information about these organizations and their digital libraries, see

ACM	http://www.acm.org
IEEE Computer Society	http://www.computer.org
USENIX	http://www.usenix.org

All operating systems researchers realize that current operating systems are massive, inflexible, unreliable, insecure, and loaded with bugs, certain ones more than others (*names withheld to protect the guilty*). Consequently, there is a lot of research on how to build better ones. Work has recently been published about bugs and debugging (Kasikci et al., 2017; Pina et al., 2019; Li et al., 2019), crashes and recovery (Chen et al., 2017; and Bhat et al., 2021), energy management (Petrucci and Loques, 2012; Shen et al., 2013; and Li et al., 2020), file and storage systems (Zhang et al., 2013a; Chen et al., 2017; Maneas et al., 2020; Ji et al., 2021; and Miller et al., 2021), high-performance I/O (Rizzo, 2012; Li et al., 2013a; and Li et al., 2017); hyperthreading and multithreading (Li et al., 2019), dynamic updates (Pina et al., 2019), managing GPUs (Volos et al., 2018), memory management (Jantz et al., 2013; and Jeong et al., 2013), embedded systems (Levy et al., 2017), operating system correctness and reliability (Klein et al., 2009; and Chen et al., 2017), operating system reliability (Chen et al., 2017; Chajed et al., 2019; and Zou et al., 2019), security (Oliverio et al., 2017; Konoth et al., 2018; Osterlund et al., 2019; Duta et al. 2021), virtualization and containers (Tack Lim et al., 2017; Manco et al., 2017; and Tarasov et al., 2013) among many other topics.

1.10 OUTLINE OF THE REST OF THIS BOOK

We have now completed our introduction and bird's-eye view of the operating system. It is time to get down to the details. As mentioned, from the programmer's point of view, the primary purpose of an operating system is to provide some